Chapitre 2 — Environnement de développement et outils

> **Partie I — Fondations du C++ moderne**
> Norme de référence : C++20 / C++23 — Compilateurs : GCC 14+, Clang 18+, MSVC 2022 (17.10+)

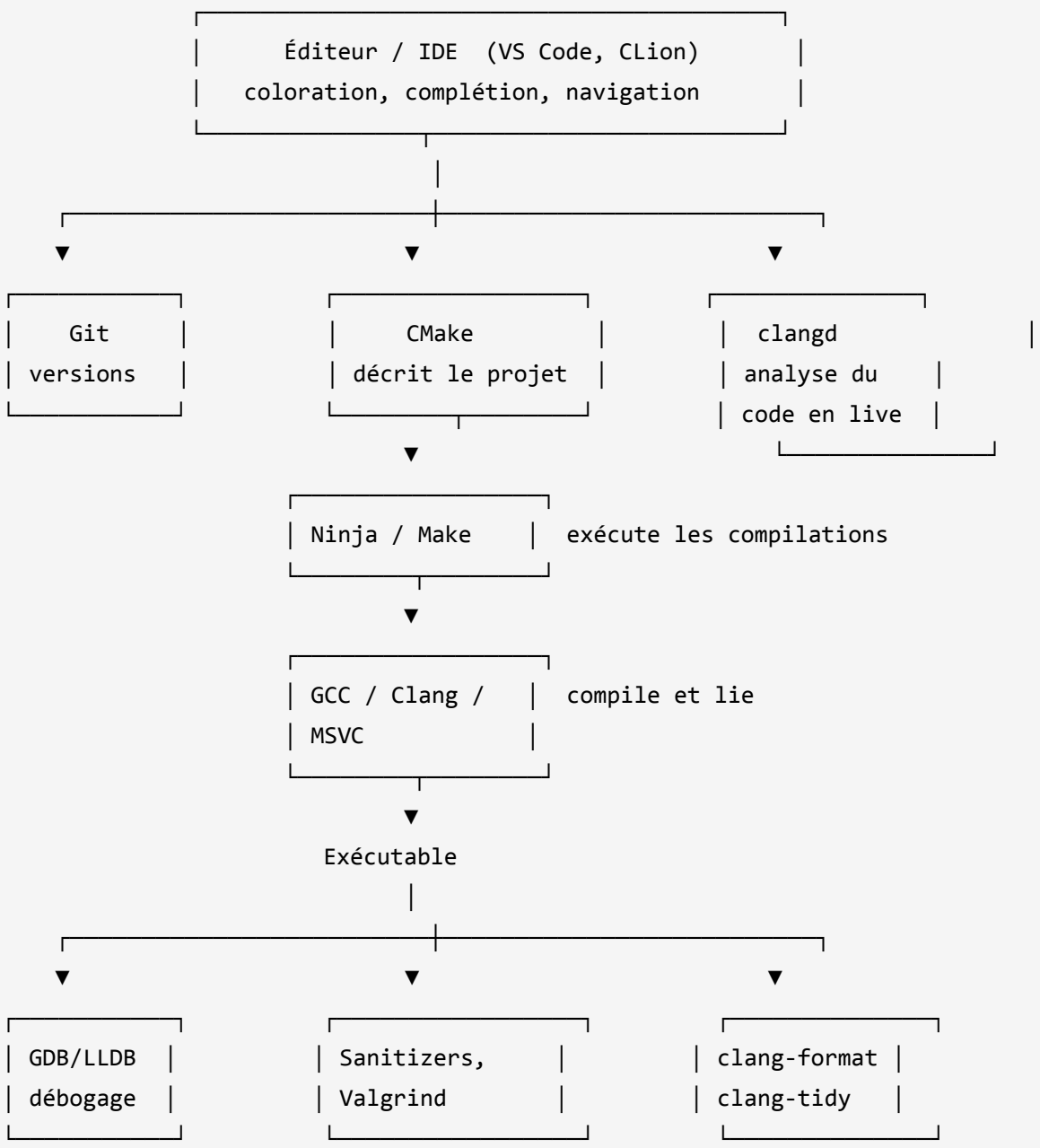
2.1 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

i. Décrire les composants d'une chaîne d'outils C++ professionnelle et le rôle de chacun.
ii. Installer et vérifier un environnement complet sous Linux, Windows ou macOS.
iii. Choisir et utiliser les options essentielles du compilateur (norme, avertissements, optimisation, débogage).
iv. Configurer un éditeur ou un IDE moderne (VS Code avec clangd, CLion, Visual Studio).
v. Écrire un fichier `CMakeLists.txt`, configurer et construire un projet avec CMake et des *presets*.
vi. Intégrer une bibliothèque externe (FetchContent, vcpkg, Conan).
vii. Déboguer un programme avec GDB ou LLDB.
viii. Détecter les erreurs mémoire et les comportements indéfinis avec les sanitizers et Valgrind.
ix. Formater et analyser automatiquement le code avec clang-format et clang-tidy.
x. Versionner un projet avec Git et le publier sur GitHub.
xi. Mettre en place le squelette du projet fil rouge UIN-Campus.

- ## 2.2 Vue d'ensemble : la chaîne d'outils (*toolchain*)

  Au chapitre 1, nous avons compilé à la main avec `g++`. Cela suffit pour trois fichiers, mais un vrai projet compte des centaines de fichiers, des bibliothèques externes, des tests, plusieurs développeurs. Il faut alors une **chaîne d'outils** complète.


Éditeur / IDE (VS Code, CLion)
coloration, complétion, navigation
Git
versions
CMake
décrit le projet
clangd
analyse du
code en live
Ninja / Make
exécute les compilations
GCC / Clang /
MSVC
compile et lie
Exécutable
GDB/LLDB
débogage
Sanitizers,
Valgrind
clang-format
clang-tidy

Catégorie	Outils	Rôle
Compilateur	GCC, Clang, MSVC	Traduire le C++ en code machine
Système de construction	CMake (+ Ninja ou Make)	Décrire le projet et orchestrer la compilation
Gestionnaire de paquets	vcpkg, Conan	Télécharger et intégrer les bibliothèques externes
Éditeur / IDE	VS Code + clangd, CLion, Visual Studio, Qt Creator	Écrire et naviguer dans le code
Débogueur	GDB, LLDB, débogueur de Visual Studio	Exécuter pas à pas, inspecter les variables
Détection d'erreurs	AddressSanitizer, UBSan, Valgrind	Trouver les erreurs mémoire et les UB
Qualité du code	clang-format, clang-tidy, cppcheck	Uniformiser le style, détecter les mauvaises pratiques
Tests	GoogleTest, Catch2	Vérifier automatiquement le code (chapitre 23)
Versions	Git, GitHub	Historique, collaboration, sauvegarde
Outils en ligne	Compiler Explorer, cppreference	Expérimenter, se documenter

---

- # 2.3 Installation de l'environnement

> **Recommandation pour ce cours** : travailler sous **Linux** (Ubuntu, Pop!_OS, Debian, Fedora…), soit en natif, soit via **WSL2** sous

> Windows. C'est l'environnement où tous les outils du cours (sanitizers, Valgrind, GDB) fonctionnent sans restriction. Les autres plateformes restent possibles, avec les limites indiquées.

- ## 2.3.1 Linux (Ubuntu 24.04, Pop!_OS, Debian et dérivés)

```
sudo apt update
sudo apt install build-essential gcc-14 g++-14 \
                 clang clangd clang-format clang-tidy lldb gdb \
                 cmake ninja-build valgrind cppcheck git
```

Sur Ubuntu 24.04 et ses dérivés, le `g++` par défaut est la version 13. Pour utiliser GCC 14 par défaut :

```
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-14 140 \
                         --slave /usr/bin/g++ g++ /usr/bin/g++-14
```

Sur Fedora, l'équivalent est :

```
sudo dnf install gcc-c++ clang clang-tools-extra lldb gdb cmake ninja-build valgrind cppcheck git
```

- ## 2.3.2 Windows

Trois options, de la plus recommandée à la moins recommandée pour ce cours :
Option A — WSL2 (recommandée). WSL2 exécute un vrai Linux dans Windows.

```
wsl --install -d Ubuntu-24.04
```

Puis, dans le terminal Ubuntu, suivre les instructions Linux ci-dessus. VS Code s'y connecte avec l'extension **WSL**.
Option B — Visual Studio 2022 Community. Lors de l'installation, cocher la charge de travail **« Développement Desktop en C++ »**. Elle installe MSVC, CMake, Ninja et le débogueur intégré. C'est l'environnement le plus confortable sous Windows natif, mais Valgrind n'existe

pas et seul AddressSanitizer est disponible (`/fsanitize=address`).

Option C — MSYS2 (GCC natif sous Windows). Après installation de MSYS2, dans le terminal **UCRT64** :

```
pacman -S --needed mingw-w64-ucrt-x86_64-toolchain \
                   mingw-w64-ucrt-x86_64-cmake \
                   mingw-w64-ucrt-x86_64-ninja \
                   mingw-w64-ucrt-x86_64-clang-tools-extra \
                   git
```

Puis ajouter `C:\msys64\ucrt64\bin` à la variable d'environnement `PATH` . Limite importante : les sanitizers ne sont pas pris en charge par GCC sous MinGW.

- ## 2.3.3 macOS

```
xcode-select --install          # compilateur Apple Clang et outils de base
brew install cmake ninja llvm   # Homebrew : LLVM récent, clangd, clang-tidy
```

Le LLVM de Homebrew est souvent plus récent qu'Apple Clang et prend mieux en charge les dernières fonctionnalités. Valgrind n'est pas disponible sur les Mac à puce Apple : on utilise AddressSanitizer à la place.

- ## 2.3.4 Vérifier l'installation

```
g++ --version
clang++ --version
cmake --version       # 3.25 minimum pour ce cours
ninja --version
gdb --version
git --version
```

Test rapide de la prise en charge de C++23 :

```
// test_cpp23.cpp
#include <print>

int main()
{
  std::println("C++23 fonctionne : __cplusplus = {}", __cplusplus);
}
```

```
g++ -std=c++23 test_cpp23.cpp -o test && ./test
```

Si la compilation échoue sur `<print>`, le compilateur est trop ancien : revenir à l'étape d'installation.

2.4 Le compilateur et ses options

2.4.1 Options essentielles de GCC et Clang

GCC et Clang acceptent pratiquement les mêmes options.

Option	Rôle
`-std=c++20`, `-std=c++23`	Choisir la norme
`-o nom`	Nom du fichier produit
`-c`	Compiler sans lier (produit un `.o`)

Option	Rôle
`-I chemin`	Ajouter un répertoire où chercher les en-têtes
`-L chemin`, `-lnom`	Répertoire des bibliothèques, et bibliothèque à lier
`-D NOM=valeur`	Définir une macro
`-Wall`	Active les avertissements les plus courants (malgré son nom, pas *tous*)
`-Wextra`	Avertissements supplémentaires
`-Wpedantic`	Signale les extensions non conformes à la norme
`-Wshadow`	Signale une variable qui en masque une autre
`-Wconversion`	Signale les conversions implicites qui peuvent perdre de l'information
`-Werror`	Transforme les avertissements en erreurs
`-O0`	Aucune optimisation (par défaut, idéal pour déboguer)
`-O2`	Optimisations standard (production)
`-O3`	Optimisations agressives
`-g`	Inclut les informations de débogage
`-fsanitize=...`	Active un sanitizer (section 2.8)

Jeu d'options recommandé pour le développement :

```
g++ -std=c++23 -Wall -Wextra -Wpedantic -Wshadow -Wconversion -g -O0 main.cpp -o main
```

- ## 2.4.2 Pourquoi les avertissements sont vos alliés

```
#include <iostream>

int main()
{
  double moyenne = 13.75;
  int arrondi = moyenne;          // perte de la partie décimale

  int notes = 5;
  {
      int notes = 10;             // masque la variable extérieure
      std::cout << notes << '\n';
  }

  unsigned int n = 3;
  int i = -1;
  if (i < n) {                    // piège : -1 converti en unsigned devient énorme
      std::cout << "jamais affiché\n";
  }
}
```

Avec les options recommandées, le compilateur signale les trois problèmes :

```
warning: conversion from 'double' to 'int' may change value [-Wfloat-conversion]
warning: declaration of 'notes' shadows a previous local [-Wshadow]
warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
```

Sans ces options, le programme compile **en silence**, et le troisième test donne un résultat faux : `-1 < 3` est évalué à `false` parce que `-1` est converti en un très grand entier non signé.

> 💡 **À retenir** : un avertissement n'est pas un détail. C'est le compilateur qui vous signale un bug probable. Dans ce cours, l'objectif est **zéro avertissement**.

2.4.3 Équivalents pour MSVC

GCC / Clang	MSVC	Rôle
`-std=c++20`	`/std:c++20`	Norme C++20
`-std=c++23`	`/std:c++latest`	Dernière norme prise en charge
`-Wall -Wextra`	`/W4`	Avertissements étendus
`-Werror`	`/WX`	Avertissements traités comme erreurs
`-Wpedantic`	`/permissive-`	Conformité stricte à la norme
`-g`	`/Zi`	Informations de débogage
`-O0` / `-O2`	`/Od` / `/O2`	Optimisation
`-fsanitize=address`	`/fsanitize=address`	AddressSanitizer
—	`/EHsc`	Modèle standard de gestion des exceptions
—	`/Zc:__cplusplus`	Valeur correcte de `__cplusplus`

2.4.4 GCC ou Clang ?

Les deux sont d'excellente qualité. Il est même recommandé de **compiler avec les deux** : chacun détecte des problèmes que l'autre ne voit pas, et un code qui compile sans avertissement sur plusieurs compilateurs est plus portable. Clang est souvent apprécié pour la clarté de ses messages d'erreur et il est à la base des outils clangd, clang-format et clang-tidy.

2.5 Éditeurs et environnements de développement

2.5.1 Comparatif

Outil	Type	Points forts	Licence
VS Code + clangd	Éditeur extensible	Léger, multiplateforme, très répandu	Gratuit
CLion (JetBrains)	IDE complet	Refactorisation puissante, intégration CMake native	Gratuit pour usage non commercial et pour l'enseignement
Visual Studio 2022	IDE complet	Meilleur débogueur sous Windows	Community gratuit
Qt Creator	IDE	Idéal pour les applications Qt	Gratuit (open source)

Ce cours utilise **VS Code avec clangd**, mais tout le code fonctionne avec n'importe quel outil, puisque le projet est décrit par CMake.

2.5.2 Configurer VS Code

Extensions à installer :

Extension	Rôle
clangd (LLVM)	Complétion, erreurs en direct, navigation, intégration de clang-tidy
CMake Tools (Microsoft)	Configurer, construire et lancer le projet CMake

Extension	Rôle
CodeLLDB	Débogage avec LLDB
WSL	Travailler dans WSL2 depuis Windows (si applicable)

> **Conflit à éviter** : l'extension Microsoft **C/C++** possède son propre moteur d'analyse (IntelliSense), qui entre en conflit avec clangd. Si elle est installée, désactivez son moteur dans les réglages :

```
{
  "C_Cpp.intelliSenseEngine": "disabled",
  "clangd.arguments": ["--header-insertion=never"],
  "editor.formatOnSave": true
}
```

- ## 2.5.3 Le fichier `compile_commands.json`

clangd a besoin de savoir **comment** chaque fichier est compilé : norme, options, répertoires d'en-têtes. Cette information se trouve dans un fichier `compile_commands.json`, que CMake génère automatiquement avec l'option :

```
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
```

Le fichier est créé dans le répertoire de construction. Pour que clangd le trouve, on crée un lien à la racine du projet :

```
ln -s build/debug/compile_commands.json .
```

> 💡 **Symptôme typique** : si clangd souligne en rouge `#include <print>` ou ne reconnaît pas les en-têtes de votre projet, c'est presque toujours que `compile_commands.json` est absent ou obsolète.

2.6 CMake : décrire et construire un projet

2.6.1 Pourquoi un système de construction ?

Compiler à la main pose rapidement des problèmes :

- il faut retaper de longues commandes ;
- il faut savoir quels fichiers recompiler après une modification ;
- les commandes diffèrent selon le compilateur et le système (GCC sous Linux, MSVC sous Windows) ;
- il faut gérer les bibliothèques externes et leurs chemins.

CMake résout ces problèmes. Point important : CMake **ne compile pas** lui-même. C'est un **générateur** : il lit une description du projet (`CMakeLists.txt`) et produit les fichiers pour un outil qui, lui, lance les compilations (**Ninja**, Make, ou un projet Visual Studio).

```
CMakeLists.txt ⟶ CMake (configuration) ⟶ build.ninja ⟶ Ninja ⟶ g++ / clang++ / cl
```

CMake est aujourd'hui le **standard de fait** de l'industrie C++ : la grande majorité des bibliothèques et des IDE le prennent en charge.

2.6.2 Premier `CMakeLists.txt`

Reprenons le projet du chapitre 1 (`moyenne.h`, `moyenne.cpp`, `main.cpp`) :

```
cmake_minimum_required(VERSION 3.25)

project(Bulletin VERSION 1.0 LANGUAGES CXX)

add_executable(bulletin main.cpp moyenne.cpp)
target_compile_features(bulletin PRIVATE cxx_std_23)
```

Commande	Rôle
`cmake_minimum_required`	Version minimale de CMake requise
`project`	Nom du projet, version, langage
`add_executable`	Déclare une **cible** : un exécutable et ses sources
`target_compile_features`	Exige une norme pour cette cible

- ## 2.6.3 Configurer et construire

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Debug   # 1. configuration
cmake --build build                                      # 2. construction
./build/bulletin                                         # 3. exécution
```

Option	Signification
`-S .`	Répertoire des sources (ici le répertoire courant)
`-B build`	Répertoire de construction
`-G Ninja`	Générateur à utiliser
`-DCMAKE_BUILD_TYPE=Debug`	Type de construction

> 💡 **Construction hors des sources** (*out-of-source build*) : tous les fichiers générés vont dans `build/`. Les sources restent propres, et l'on peut tout effacer sans risque avec `rm -rf build`.

Après une modification, seule la commande `cmake --build build` est nécessaire : CMake et Ninja ne recompilent que les fichiers

concernés.

- ## 2.6.4 Les types de construction

Type	Options équivalentes (GCC)	Usage
`Debug`	`-O0 -g`	Développement et débogage
`Release`	`-O3 -DNDEBUG`	Version finale, la plus rapide
`RelWithDebInfo`	`-O2 -g -DNDEBUG`	Rapide, mais encore débogable
`MinSizeRel`	`-Os -DNDEBUG`	Exécutable le plus petit possible

La macro `NDEBUG` désactive les `assert` (chapitre 15).

- ## 2.6.5 La notion de cible (*target*)

Le CMake moderne est organisé autour des **cibles** : exécutables et bibliothèques. Chaque propriété (norme, options, répertoires d'en-têtes, dépendances) est attachée **à une cible**, avec une portée :

Portée	Signification
`PRIVATE`	Utile seulement pour construire cette cible
`PUBLIC`	Utile pour cette cible **et** pour celles qui l'utilisent
`INTERFACE`	Utile seulement pour celles qui l'utilisent

Exemple : une bibliothèque de calcul utilisée par un exécutable.

```cmake
cmake_minimum_required(VERSION 3.25)
project(Bulletin VERSION 1.0 LANGUAGES CXX)

set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Bibliothèque
add_library(calcul src/moyenne.cpp)
target_include_directories(calcul PUBLIC ${PROJECT_SOURCE_DIR}/include)
target_compile_features(calcul PUBLIC cxx_std_23)

# Exécutable qui utilise la bibliothèque
add_executable(bulletin app/main.cpp)
target_link_libraries(bulletin PRIVATE calcul)
```

Grâce à `PUBLIC`, l'exécutable `bulletin` hérite automatiquement du répertoire `include/` et de l'exigence C++23 de `calcul`. Il n'y a rien à répéter.

> **CMake ancien à éviter** : on trouve encore dans de nombreux tutoriels `include_directories(...)`, `add_definitions(...)` ou `set(CMAKE_CXX_FLAGS "...")`. Ces commandes s'appliquent **globalement** à tout le projet et rendent les grands projets difficiles à maintenir. Préférez toujours les commandes `target_...`.

- ## 2.6.6 Options de compilation selon le compilateur

On utilise une **bibliothèque d'interface** pour regrouper les options communes et les partager entre toutes les cibles :

```
add_library(options_projet INTERFACE)
target_compile_features(options_projet INTERFACE cxx_std_23)

if(MSVC)
  target_compile_options(options_projet INTERFACE /W4 /permissive-)
else()
  target_compile_options(options_projet INTERFACE
      -Wall -Wextra -Wpedantic -Wshadow -Wconversion)
endif()

target_link_libraries(calcul   PRIVATE options_projet)
target_link_libraries(bulletin PRIVATE options_projet)
```

- ## 2.6.7 Les *presets* CMake

Retaper `-G Ninja -DCMAKE_BUILD_TYPE=Debug ...` est fastidieux et source d'erreurs. Le fichier **`CMakePresets.json`**, placé à la racine du projet, enregistre des configurations nommées :

```
{
  "version": 6,
  "configurePresets": [
      {
          "name": "debug",
          "displayName": "Debug",
          "generator": "Ninja",
          "binaryDir": "${sourceDir}/build/debug",
          "cacheVariables": {
              "CMAKE_BUILD_TYPE": "Debug"
          }
      },
      {
          "name": "release",
          "displayName": "Release",
          "generator": "Ninja",
          "binaryDir": "${sourceDir}/build/release",
          "cacheVariables": {
              "CMAKE_BUILD_TYPE": "Release"
          }
      }
  ],
  "buildPresets": [
      { "name": "debug",   "configurePreset": "debug" },
      { "name": "release", "configurePreset": "release" }
  ]
}
```

Utilisation :

```
cmake --list-presets
cmake --preset debug
cmake --build --preset debug
```

VS Code (CMake Tools), CLion et Visual Studio reconnaissent automatiquement ce fichier.

2.6.8 Récapitulatif des commandes CMake essentielles

Commande	Rôle
`add_executable(nom sources...)`	Créer un exécutable
`add_library(nom sources...)`	Créer une bibliothèque (statique par défaut)
`add_library(nom INTERFACE)`	Bibliothèque sans sources, porteuse d'options
`target_include_directories`	Répertoires d'en-têtes
`target_link_libraries`	Dépendances
`target_compile_features`	Norme exigée
`target_compile_options`	Options du compilateur
`target_compile_definitions`	Macros (`-D`)
`option(NOM "desc" OFF)`	Option activable par l'utilisateur
`add_subdirectory(dossier)`	Inclure un sous-projet ayant son propre `CMakeLists.txt`
`enable_testing()` , `add_test`	Tests (chapitre 23)

2.7 Utiliser des bibliothèques externes

La bibliothèque standard ne couvre pas tout : lecture de JSON, interfaces graphiques, réseau, tests unitaires… Il existe trois manières courantes d'intégrer une bibliothèque externe.

2.7.1 FetchContent (intégré à CMake)

CMake télécharge la bibliothèque et la construit avec le projet. Exemple avec **nlohmann/json**, que nous utiliserons au chapitre 20 :

```
include(FetchContent)

FetchContent_Declare(
  json
  URL https://github.com/nlohmann/json/releases/download/v3.11.3/json.tar.xz
)
FetchContent_MakeAvailable(json)

target_link_libraries(bulletin PRIVATE nlohmann_json::nlohmann_json)
```

Avantage : aucune installation préalable. Inconvénient : le téléchargement se fait à la configuration, et une connexion Internet est nécessaire la première fois.

2.7.2 vcpkg (Microsoft)

vcpkg est un gestionnaire de paquets C++ multiplateforme. En **mode manifeste**, on déclare les dépendances dans un fichier `vcpkg.json` à la racine :

```
{
  "name": "bulletin",
  "version": "1.0.0",
  "dependencies": [ "nlohmann-json" ]
}
```

On indique ensuite à CMake d'utiliser vcpkg :

```
cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=<chemin_vcpkg>/scripts/buildsystems/vcpkg.cmake
```

Dans le `CMakeLists.txt` :

```
find_package(nlohmann_json CONFIG REQUIRED)
target_link_libraries(bulletin PRIVATE nlohmann_json::nlohmann_json)
```

- ## 2.7.3 Conan

Conan (version 2) est un autre gestionnaire de paquets, très utilisé dans l'industrie. Fichier `conanfile.txt` :

```
[requires]
nlohmann_json/3.11.3

[generators]
CMakeDeps
CMakeToolchain
```

```
conan install . --output-folder=build --build=missing
cmake -S . -B build -DCMAKE_TOOLCHAIN_FILE=build/conan_toolchain.cmake
```

2.7.4 Quelle méthode choisir ?

Méthode	Quand l'utiliser
FetchContent	Petites bibliothèques, projets pédagogiques, peu de dépendances
vcpkg	Projets multiplateformes, surtout avec Visual Studio
Conan	Projets industriels avec beaucoup de dépendances et de configurations
Paquets système (`apt install libxxx-dev`)	Environnement Linux maîtrisé, sans besoin de portabilité

Dans ce cours, nous utiliserons **FetchContent** : il ne demande rien de plus que CMake.

2.8 Le débogage avec GDB et LLDB

2.8.1 Qu'est-ce qu'un débogueur ?

Un débogueur permet d'**exécuter un programme sous contrôle** : l'arrêter à une ligne précise, avancer instruction par instruction, examiner et modifier les variables, remonter la pile des appels. C'est un outil beaucoup plus efficace que l'ajout de `std::cout` partout dans le code.
Deux conditions pour bien déboguer :

- compiler avec `-g` (informations de débogage) ;
- compiler avec `-O0` (sans optimisation), sinon le compilateur réorganise le code et certaines variables deviennent invisibles.
  C'est exactement ce que fait le type de construction `Debug` de CMake.

2.8.2 Un programme à déboguer

```cpp
// bug.cpp
#include <iostream>
#include <vector>

double moyenne(const std::vector<double>& notes)
{
  double somme = 0.0;
  for (std::size_t i = 0; i <= notes.size(); ++i) {
      somme += notes[i];
  }
  return somme / static_cast<double>(notes.size());
}

int main()
{
  std::vector<double> notes{12.0, 14.0, 16.0};
  std::cout << "Moyenne : " << moyenne(notes) << '\n';   // attendu : 14
}
```

Le programme affiche une valeur fausse (ou parfois la bonne, par hasard). Cherchons pourquoi.

- ## 2.8.3 Session GDB

```
g++ -std=c++23 -g -O0 bug.cpp -o bug
gdb ./bug
```

```
(gdb) break moyenne             # point d'arrêt à l'entrée de la fonction
Breakpoint 1 at ...: file bug.cpp, line 7.
(gdb) run                       # lancer le programme
Breakpoint 1, moyenne (notes=std::vector of length 3, capacity 3 = {...}) at bug.cpp:7
7           double somme = 0.0;
(gdb) print notes               # afficher le vecteur
$1 = std::vector of length 3, capacity 3 = {12, 14, 16}
(gdb) next                      # ligne suivante
8           for (std::size_t i = 0; i <= notes.size(); ++i) {
(gdb) display i                 # afficher i à chaque arrêt
(gdb) display somme
(gdb) next
...
1: i = 3
2: somme = 42
9               somme += notes[i];
```

Avec `i = 3`, le programme va lire `notes[3]` alors que le vecteur n'a que 3 éléments (indices 0 à 2). La condition `i <= notes.size()` doit être `i < notes.size()`. **Bug trouvé.**

- ## 2.8.4 Commandes essentielles

GDB	LLDB	Rôle
`run` (`r`)	`run`	Lancer le programme
`break f` (`b`)	`b f`	Point d'arrêt sur la fonction `f`
`break fichier.cpp:12`	`b fichier.cpp:12`	Point d'arrêt sur une ligne

GDB	LLDB	Rôle
`next` (`n`)	`next` (`n`)	Ligne suivante, **sans entrer** dans les fonctions appelées
`step` (`s`)	`step` (`s`)	Ligne suivante, **en entrant** dans les fonctions
`finish`	`finish`	Terminer la fonction courante et revenir à l'appelant
`continue` (`c`)	`continue` (`c`)	Reprendre jusqu'au prochain point d'arrêt
`print x` (`p`)	`p x`	Afficher une variable ou une expression
`display x`	`display x`	Afficher `x` à chaque arrêt
`info locals`	`frame variable`	Toutes les variables locales
`backtrace` (`bt`)	`bt`	Pile des appels
`frame 1`	`frame select 1`	Se placer dans un autre niveau de la pile
`watch x`	`watchpoint set variable x`	S'arrêter quand `x` change
`info breakpoints`	`breakpoint list`	Lister les points d'arrêt
`delete 1`	`breakpoint delete 1`	Supprimer un point d'arrêt
`quit` (`q`)	`quit`	Quitter

- ## 2.8.5 La pile des appels (*backtrace*)

Quand un programme plante, la commande `backtrace` montre **le chemin qui a mené au plantage** :

```
(gdb) bt
#0  moyenne (notes=...) at bug.cpp:9
#1  0x0000555555555320 in main () at bug.cpp:16
```

On lit de haut en bas : le plantage a eu lieu dans `moyenne` (ligne 9), appelée depuis `main` (ligne 16).

- ## 2.8.6 Déboguer dans VS Code

Avec l'extension CodeLLDB, créer le fichier `.vscode/launch.json` :

```
{
  "version": "0.2.0",
  "configurations": [
      {
          "name": "Déboguer",
          "type": "lldb",
          "request": "launch",
          "program": "${workspaceFolder}/build/debug/uin-campus",
          "args": [],
          "cwd": "${workspaceFolder}"
      }
  ]
}
```

On place ensuite les points d'arrêt en cliquant dans la marge, et on lance avec **F5**. Les variables, la pile d'appels et les expressions surveillées s'affichent dans le panneau latéral. Les mêmes commandes que dans GDB sont disponibles sous forme de boutons.

2.9 Détecter les erreurs : sanitizers et Valgrind

Le débogueur aide à comprendre un bug **qu'on a remarqué**. Mais beaucoup d'erreurs mémoire et de comportements indéfinis passent **inaperçus** : le programme semble fonctionner. Les sanitizers et Valgrind les détectent automatiquement.

2.9.1 Les sanitizers

Un sanitizer est une instrumentation ajoutée par le compilateur : le code produit contient des vérifications supplémentaires qui s'exécutent pendant le programme.

Sanitizer	Option	Détecte
AddressSanitizer (ASan)	`-fsanitize=address`	Accès hors limites (pile, tas, global), utilisation après libération, double libération
LeakSanitizer (LSan)	inclus dans ASan sous Linux	Fuites mémoire
UndefinedBehaviorSanitizer (UBSan)	`-fsanitize=undefined`	Débordement d'entier signé, division par zéro entière, décalages invalides, déréférencement de pointeur nul…
ThreadSanitizer (TSan)	`-fsanitize=thread`	Conditions de course entre threads (chapitre 24)
MemorySanitizer (MSan)	`-fsanitize=memory` (Clang seulement)	Lecture de mémoire non initialisée

Utilisation recommandée pendant le développement :

```
g++ -std=c++23 -g -O1 -fno-omit-frame-pointer -fsanitize=address,undefined bug.cpp -o bug
./bug
```

L'option `-fno-omit-frame-pointer` améliore la précision des rapports.

> **Incompatibilités** : ASan et TSan ne peuvent pas être activés ensemble ; MSan non plus. En revanche, ASan et UBSan se combinent sans problème.

- ## 2.9.2 Lire un rapport d'AddressSanitizer

Sur le programme `bug.cpp` :

```
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x503000000058
READ of size 8 at 0x503000000058 thread T0
  #0 0x... in moyenne(std::vector<double> const&) bug.cpp:9
  #1 0x... in main bug.cpp:16

0x503000000058 is located 0 bytes after 24-byte region [0x503000000040,0x503000000058)
allocated by thread T0 here:
  ...
SUMMARY: AddressSanitizer: heap-buffer-overflow bug.cpp:9 in moyenne(...)
```

Comment lire ce rapport :

Élément	Signification
`heap-buffer-overflow`	Type d'erreur : dépassement d'une zone allouée sur le tas
`READ of size 8`	Lecture de 8 octets, soit un `double`
`bug.cpp:9`	Ligne fautive
`0 bytes after 24-byte region`	Juste après la fin d'une zone de 24 octets (3 × 8 octets = les 3 notes)

Le diagnostic est immédiat et précis, alors que sans sanitizer le programme affichait simplement une valeur fausse.

- ## 2.9.3 Exemples pour UBSan et LeakSanitizer

Débordement d'entier signé :

```
#include <iostream>
#include <limits>

int main()
{
  int x = std::numeric_limits<int>::max();
  int increment{};
  std::cin >> increment;      // saisir 1
  x += increment;
  std::cout << x << '\n';
}
```

```
runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
```

Fuite mémoire :

```
int main()
{
  int* notes = new int[10];
  notes[0] = 15;
  return 0;                   // delete[] oublié
}
```

```
==12346==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 40 byte(s) in 1 object(s) allocated from:
  #0 ... in operator new[](unsigned long)
  #1 ... in main fuite.cpp:3
```

- ## 2.9.4 Valgrind

Valgrind (outil *memcheck*) détecte des erreurs similaires, mais par une autre méthode : il exécute le programme dans une machine virtuelle qui surveille chaque accès mémoire. Il n'est **pas nécessaire de recompiler** avec une option spéciale (seulement `-g` pour avoir les numéros de ligne).

```
g++ -std=c++23 -g -O0 fuite.cpp -o fuite
valgrind --leak-check=full ./fuite
```

```
==12347== HEAP SUMMARY:
==12347==     in use at exit: 40 bytes in 1 blocks
==12347== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12347==    at 0x...: operator new[](unsigned long)
==12347==    by 0x...: main (fuite.cpp:3)
```

- ## 2.9.5 Sanitizers ou Valgrind ?

Critère	Sanitizers	Valgrind
Recompilation nécessaire	Oui	Non
Ralentissement	Environ ×2	Environ ×20 à ×50

Critère	Sanitizers	Valgrind
Débordements sur la pile	Détectés	Mal détectés
Plateformes	Linux, macOS, Windows (ASan avec MSVC)	Linux surtout
Programme sans les sources	Impossible	Possible

> 💡 **Recommandation** : activer ASan et UBSan en permanence dans la configuration Debug de vos projets. Utiliser Valgrind en complément, ou quand on ne peut pas recompiler. Ne pas utiliser Valgrind sur un programme déjà compilé avec ASan : les deux entrent en conflit.

- ## 2.9.6 Intégrer les sanitizers dans CMake

```
option(ACTIVER_SANITIZERS "Activer ASan et UBSan" OFF)

if(ACTIVER_SANITIZERS AND NOT MSVC)
  target_compile_options(options_projet INTERFACE
      -fsanitize=address,undefined -fno-omit-frame-pointer)
  target_link_options(options_projet INTERFACE
      -fsanitize=address,undefined)
endif()
```

On l'active dans le *preset* Debug avec `"ACTIVER_SANITIZERS": "ON"` dans `cacheVariables`.

- # 2.10 Qualité du code : clang-format et clang-tidy

- ## 2.10.1 clang-format : un style uniforme, automatiquement

  Dans une équipe, chacun a ses habitudes : accolades, espaces, indentation. **clang-format** reformate automatiquement le code selon des règles définies dans un fichier `.clang-format` à la racine du projet. On ne débat plus du style : l'outil l'applique.
  Exemple de configuration (celle utilisée dans ce cours) :

  ```
  BasedOnStyle: LLVM
  IndentWidth: 4
  ColumnLimit: 100
  PointerAlignment: Left
  BreakBeforeBraces: Custom
  BraceWrapping:
  AfterFunction: true
  AfterClass: false
  AfterStruct: false
  AfterControlStatement: Never
  AllowShortFunctionsOnASingleLine: Inline
  ```

  Utilisation :

  ```
  clang-format -i src/*.cpp include/**/*.hpp     # -i : modifie les fichiers sur place
  ```

  Dans VS Code, avec `"editor.formatOnSave": true`, le formatage s'applique à chaque enregistrement.
  Avant :

  ```
  double moyenne(const std::vector<double> &notes){double s=0;for(double n:notes)s+=n;
  return s/notes.size();}
  ```

  Après :

```
double moyenne(const std::vector<double>& notes)
{
  double s = 0;
  for (double n : notes)
      s += n;
  return s / notes.size();
}
```

- ## 2.10.2 clang-tidy : un relecteur automatique

clang-tidy va plus loin que le compilateur : il détecte des erreurs probables, des pratiques dépassées et des problèmes de performance, et propose souvent une correction automatique. Ses vérifications sont regroupées par familles :

Famille	Rôle	Exemple de vérification
`bugprone-*`	Bugs probables	Division entière dans un contexte flottant
`modernize-*`	Moderniser le code	Remplacer `NULL` par `nullptr`
`performance-*`	Performance	Copie inutile d'un objet dans une boucle
`readability-*`	Lisibilité	Condition inutilement complexe
`cppcoreguidelines-*`	Règles des C++ Core Guidelines	Variable non initialisée

Fichier `.clang-tidy` à la racine du projet :

```
Checks: >
-*,
bugprone-*,
modernize-*,
performance-*,
readability-*,
cppcoreguidelines-*,
-modernize-use-trailing-return-type,
-readability-magic-numbers,
-cppcoreguidelines-avoid-magic-numbers,
-readability-identifier-length
HeaderFilterRegex: '.*'
```

Le `-*` désactive tout, puis on active les familles voulues ; les lignes commençant par `-` désactivent les vérifications jugées trop bruyantes pour un contexte pédagogique.
Exemple :

```
int* p = NULL;
for (int i = 0; i < (int)v.size(); i++) { total += v[i]; }
```

```
warning: use nullptr [modernize-use-nullptr]
warning: use range-based for loop instead [modernize-loop-convert]
warning: C-style casts are discouraged; use static_cast [cppcoreguidelines-pro-type-cstyle-cast]
```

Utilisation en ligne de commande :

```
clang-tidy -p build/debug src/moyenne.cpp          # analyse
clang-tidy -p build/debug --fix src/moyenne.cpp    # analyse et correction automatique
```

L'option `-p` indique où trouver `compile_commands.json`. clangd exécute aussi clang-tidy **en direct** dans l'éditeur : les avertissements apparaissent pendant la frappe.
On peut enfin demander à CMake de lancer clang-tidy à chaque compilation :

```
set(CMAKE_CXX_CLANG_TIDY clang-tidy)
```

2.10.3 cppcheck

cppcheck est un autre analyseur statique, indépendant de Clang, qui trouve parfois des problèmes différents :

```
cppcheck --enable=all --std=c++20 --suppress=missingIncludeSystem src/
```

2.11 Outils en ligne : Compiler Explorer et cppreference

2.11.1 Compiler Explorer (godbolt.org)

Compiler Explorer permet de compiler du code directement dans le navigateur, avec des dizaines de compilateurs et de versions. Il est idéal pour :

- tester rapidement une idée sans créer de projet ;
- vérifier si une fonctionnalité est prise en charge par un compilateur donné ;
- **voir le code assembleur produit**, et constater par exemple que `std::accumulate` produit le même code qu'une boucle écrite à la main (abstraction à coût nul, chapitre 1) ;
- partager un exemple par un lien.
  Mode d'emploi : saisir le code à gauche, choisir le compilateur (par exemple `x86-64 gcc 14`) et les options (`-std=c++23 -O2`) à droite. Le bouton **« Add new → Execution Only »** permet d'exécuter le programme.

2.11.2 cppreference.com

cppreference.com est la documentation de référence du langage et de la bibliothèque standard, maintenue par la communauté et très proche du texte de la norme. Une page type, par exemple celle de `std::vector`, comporte :

Section	Contenu
En-tête	Le fichier à inclure (`<vector>`) et la déclaration
Tableau des membres	Toutes les fonctions membres, avec la version de la norme qui les a introduites (`(since C++20)`)
Complexité	Coût algorithmique de chaque opération
Invalidation des itérateurs	Quand les itérateurs deviennent invalides (chapitre 18)
Exemple	Programme complet, exécutable en ligne
Voir aussi	Liens vers les éléments apparentés

> 💡 **Conseil** : prenez l'habitude de consulter cppreference **avant** les forums. Les réponses anciennes des forums décrivent souvent du C++ dépassé.

2.12 Versionner avec Git et GitHub

2.12.1 Pourquoi versionner ?

Un système de gestion de versions enregistre l'**historique complet** d'un projet. Il permet de revenir à une version antérieure, de savoir qui a modifié quoi et pourquoi, de travailler à plusieurs sans s'écraser mutuellement, et de disposer d'une sauvegarde distante. **Git** est aujourd'hui l'outil universel ; **GitHub** (ou GitLab) héberge les dépôts en ligne.

2.12.2 Configuration initiale (une seule fois)

```
git config --global user.name  "Prénom Nom"
git config --global user.email "adresse@exemple.com"
git config --global init.defaultBranch main
```

2.12.3 Les trois zones de Git

```
Répertoire de travail  ──git add──▶  Zone de préparation  ──git commit──▶  Dépôt (historique)
 (fichiers modifiés)                 (staging area)                        (commits)
```

2.12.4 Commandes essentielles

Commande	Rôle
`git init`	Créer un dépôt dans le répertoire courant
`git status`	État des fichiers (modifiés, préparés, non suivis)
`git add fichier` / `git add .`	Préparer des modifications
`git commit -m "message"`	Enregistrer un instantané
`git log --oneline`	Historique résumé
`git diff`	Modifications non encore préparées
`git branch nom` / `git switch nom`	Créer une branche / changer de branche
`git switch -c nom`	Créer une branche et s'y placer

Commande	Rôle
`git merge nom`	Fusionner une branche dans la branche courante
`git remote add origin URL`	Associer un dépôt distant
`git push -u origin main`	Envoyer les commits vers le dépôt distant
`git pull`	Récupérer les modifications distantes
`git clone URL`	Copier un dépôt distant
`git restore fichier`	Annuler les modifications non préparées d'un fichier

- ## 2.12.5 Le fichier `.gitignore` pour un projet C++

On ne versionne **jamais** les fichiers générés : ils se reconstruisent à partir des sources.

```
# Répertoires de construction
build/
cmake-build-*/
out/

# Fichiers générés
compile_commands.json
*.o
*.obj
*.a
*.lib
*.so
*.dll
*.exe

# Caches des outils
.cache/
.idea/
```

- ## 2.12.6 Bonnes pratiques pour les messages de commit

Un bon message décrit **ce qui change et pourquoi**, à l'impératif et de manière concise :

À éviter	Préférer
`modif`	`Ajouter le calcul de la moyenne pondérée`
`correction bug`	`Corriger le dépassement d'indice dans moyenne()`
`trucs`	`Configurer clang-format et clang-tidy`

> 💡 Les étudiants peuvent bénéficier gratuitement d'outils professionnels grâce au **GitHub Student Developer Pack**.

- # 2.13 Structure recommandée d'un projet C++

```
mon-projet/
├── CMakeLists.txt          # description du projet
├── CMakePresets.json       # configurations prédéfinies
├── .clang-format           # règles de style
├── .clang-tidy             # règles d'analyse
├── .gitignore
├── README.md               # présentation, instructions de construction
├── include/
│   └── monprojet/          # en-têtes publics (.hpp)
├── src/                    # implémentations (.cpp)
├── app/                    # programme principal (main.cpp)
├── tests/                  # tests unitaires (chapitre 23)
└── docs/                   # documentation
```

Placer les en-têtes publics dans un sous-répertoire portant le nom du projet (`include/monprojet/`) permet d'écrire `#include "monprojet/etudiant.hpp"`, ce qui évite les conflits de noms avec d'autres bibliothèques.

> **Convention d'extensions** : ce cours utilise `.hpp` pour les en-têtes et `.cpp` pour les sources. On rencontre aussi `.h`, `.hh`, `.cc` ou `.cxx` : ce ne sont que des conventions, mais il faut en choisir une et s'y tenir.

2.14 Erreurs fréquentes

Erreur	Conséquence	Correction
Compiler sans `-g` puis lancer le débogueur	Pas de numéros de ligne ni de noms de variables	Construire en `Debug`
Déboguer une version `-O2`	Variables « optimisées », exécution désordonnée	Utiliser `-O0` pour déboguer
Construire dans le répertoire des sources	Sources mélangées aux fichiers générés	Toujours `-B build`
Utiliser `include_directories` et `CMAKE_CXX_FLAGS` globaux	Projet difficile à maintenir	Commandes `target_...`
clangd affiche de fausses erreurs	Pas de `compile_commands.json`	`CMAKE_EXPORT_COMPILE_COMMANDS ON` et lien symbolique
Extensions C/C++ Microsoft et clangd actives ensemble	Erreurs en double, lenteurs	Désactiver IntelliSense
Versionner le répertoire `build/`	Dépôt énorme, conflits permanents	`.gitignore`
Lancer Valgrind sur un binaire ASan	Rapports incohérents ou échec	Utiliser l'un ou l'autre
Ignorer les rapports des sanitizers « parce que le résultat est bon »	Bug caché	Tout rapport de sanitizer est un bug à corriger

---

2.15 Bonnes pratiques du chapitre

i. **Décrivez chaque projet avec CMake**, même petit, en style moderne (cibles et `target_...`).
ii. **Utilisez des presets** : `debug` avec sanitizers, `release` optimisé.
iii. **Compilez avec un jeu d'avertissements strict** et visez zéro avertissement.
iv. **Activez ASan et UBSan en Debug** en permanence.
v. **Apprenez le débogueur** plutôt que de multiplier les `std::cout`.
vi. **Automatisez le style** avec clang-format et **la relecture** avec clang-tidy.
vii. **Versionnez tout projet dès le premier jour**, avec des commits fréquents et des messages clairs.
viii. **Testez avec deux compilateurs** (GCC et Clang) quand c'est possible.

2.16 Résumé du chapitre

- Une **chaîne d'outils** C++ comprend un compilateur, un système de construction, un éditeur, un débogueur, des outils de détection d'erreurs et d'analyse, et un système de versions.
- Les options du compilateur fixent la **norme** (`-std=c++23`), les **avertissements** (`-Wall -Wextra -Wpedantic -Wshadow -Wconversion`), l'**optimisation** (`-O0`, `-O2`) et le **débogage** (`-g`).
- **CMake** est un générateur : il décrit le projet en **cibles** et produit les fichiers pour **Ninja** ou un IDE. Les **presets** enregistrent les configurations.
- Les bibliothèques externes s'intègrent avec **FetchContent**, **vcpkg** ou **Conan**.
- **GDB** et **LLDB** permettent d'exécuter un programme pas à pas et d'inspecter son état.
- **ASan**, **UBSan** et **Valgrind** détectent automatiquement des erreurs invisibles à l'œil nu.
- **clang-format** uniformise le style, **clang-tidy** détecte les mauvaises pratiques.
- **Git** conserve l'historique du projet, **GitHub** l'héberge.

2.17 Glossaire bilingue du chapitre

Français	Anglais	Définition courte
Analyse statique	Static analysis	Examen du code sans l'exécuter
Avertissement	Warning	Message signalant un problème probable, sans bloquer la compilation
Branche	Branch	Ligne de développement parallèle dans Git
Chaîne d'outils	Toolchain	Ensemble des outils utilisés pour produire un programme
Cible	Target	Exécutable ou bibliothèque décrit dans CMake
Construction	Build	Processus complet de production de l'exécutable
Construction hors des sources	Out-of-source build	Fichiers générés placés hors du répertoire des sources
Débogueur	Debugger	Outil pour exécuter un programme sous contrôle
Dépôt	Repository	Projet versionné avec Git
Fuite mémoire	Memory leak	Mémoire allouée jamais libérée
Générateur	Generator	Outil pour lequel CMake produit des fichiers (Ninja, Make…)
Gestionnaire de paquets	Package manager	Outil qui installe les bibliothèques externes
Instantané, validation	Commit	Enregistrement d'un état du projet dans Git
Pile des appels	Call stack / backtrace	Suite des fonctions en cours d'exécution

Français	Anglais	Définition courte
Point d'arrêt	Breakpoint	Endroit où le débogueur suspend l'exécution
Point de surveillance	Watchpoint	Arrêt quand une variable change
Préréglage	Preset	Configuration CMake nommée
Zone de préparation	Staging area	Modifications sélectionnées pour le prochain commit

---

- # 2.18 Exercices corrigés

- ## Exercice 1 — Questions de cours

i. CMake compile-t-il le code ? Expliquez son rôle exact.
ii. Quelle est la différence entre les types de construction `Debug` et `Release` ?
iii. Que signifient `PRIVATE` et `PUBLIC` dans `target_link_libraries` ?
iv. Quelle est la différence entre clang-format et clang-tidy ?
v. À quoi sert le fichier `compile_commands.json` ?
vi. Pourquoi ne faut-il pas versionner le répertoire `build/` ?

Corrigé

i. Non. CMake est un **générateur** : il lit `CMakeLists.txt` et produit les fichiers d'un outil de construction (Ninja, Make, projet Visual Studio), qui appelle ensuite le compilateur.
ii. `Debug` compile sans optimisation et avec les informations de débogage (`-O0 -g`) : le programme est lent mais facile à déboguer. `Release` optimise (`-O3`) et définit `NDEBUG` : le programme est rapide mais difficile à déboguer et les `assert` sont désactivés.
iii. `PRIVATE` : la dépendance ne sert qu'à construire cette cible. `PUBLIC` : elle sert aussi aux cibles qui dépendent de celle-ci, qui héritent de

ses répertoires d'en-têtes, options et dépendances.

iv. clang-format ne modifie que la **présentation** (espaces, accolades, indentation), sans changer le sens du code. clang-tidy **analyse le sens** du code pour détecter des bugs, des pratiques dépassées et des problèmes de performance.

v. Il décrit, pour chaque fichier source, la commande de compilation exacte. Les outils comme clangd et clang-tidy en ont besoin pour analyser le code comme le compilateur.

vi. Il ne contient que des fichiers générés, reconstruits à partir des sources. Le versionner alourdirait le dépôt et provoquerait des conflits, puisque ces fichiers dépendent de la machine et du compilateur de chacun.

- # Exercice 2 — Quel outil utiliser ?

Pour chaque situation, indiquez l'outil le plus adapté.

i. Le programme plante et vous voulez savoir quelle suite d'appels a mené au plantage.

ii. Vous soupçonnez une fuite mémoire dans un programme dont vous n'avez que l'exécutable.

iii. Un camarade indente avec des tabulations, vous avec 4 espaces, et les fusions Git deviennent illisibles.

iv. Vous voulez savoir si `std::ranges::to` est disponible dans GCC 13.

v. Votre code utilise encore `NULL` et des casts à la C, et vous voulez les repérer automatiquement.

vi. Une boucle produit un résultat faux, sans plantage, et vous voulez suivre l'évolution d'une variable.

Corrigé

i. Le débogueur (GDB ou LLDB), commande `backtrace`.

ii. **Valgrind**, qui fonctionne sans recompilation.

iii. **clang-format**, avec un fichier `.clang-format` commun versionné dans le dépôt.

iv. **cppreference** (page *Compiler support*) ou **Compiler Explorer** pour tester directement.

v. **clang-tidy**, familles `modernize-*` et `cppcoreguidelines-*`.

vi. Le débogueur, avec `display` ou un point de surveillance (`watch`).

- # Exercice 3 — Écrire un `CMakeLists.txt`

Organisez le projet du chapitre 1 selon la structure recommandée (`include/`, `src/`, `app/`) et écrivez son `CMakeLists.txt` avec une bibliothèque `calcul`, un exécutable `bulletin`, une norme C++23 et des avertissements stricts pour GCC/Clang et MSVC.

Corrigé

Structure :

```
bulletin/
├── CMakeLists.txt
├── include/bulletin/moyenne.hpp
├── src/moyenne.cpp
└── app/main.cpp
```

Dans `src/moyenne.cpp` et `app/main.cpp`, l'inclusion devient `#include "bulletin/moyenne.hpp"`.

```
cmake_minimum_required(VERSION 3.25)
project(Bulletin VERSION 1.0 LANGUAGES CXX)

set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

add_library(options_projet INTERFACE)
target_compile_features(options_projet INTERFACE cxx_std_23)
if(MSVC)
  target_compile_options(options_projet INTERFACE /W4 /permissive-)
else()
  target_compile_options(options_projet INTERFACE
      -Wall -Wextra -Wpedantic -Wshadow -Wconversion)
endif()

add_library(calcul src/moyenne.cpp)
target_include_directories(calcul PUBLIC ${PROJECT_SOURCE_DIR}/include)
target_link_libraries(calcul PRIVATE options_projet)

add_executable(bulletin app/main.cpp)
target_link_libraries(bulletin PRIVATE calcul options_projet)
```

> Remarque : avec `-Wconversion`, la ligne `return somme / notes.size();` de `moyenne.cpp` produit un avertissement (conversion de `std::size_t` vers `double`). La correction est `somme / static_cast<double>(notes.size())`. C'est un bon exemple d'avertissement utile : il rend la conversion **visible et volontaire**.

- # Exercice 4 — Déboguer

Le programme suivant devrait afficher `Moyenne : 13.5`, mais il affiche `Moyenne : 13`. Trouvez le bug à l'aide de GDB, en précisant les commandes utilisées.

```cpp
#include <iostream>
#include <vector>

int main()
{
  std::vector<int> notes{12, 15};
  int somme = 0;
  for (int n : notes) {
      somme += n;
  }
  double moyenne = somme / notes.size();
  std::cout << "Moyenne : " << moyenne << '\n';
}
```

Corrigé

```
$ g++ -std=c++23 -g -O0 ex4.cpp -o ex4
$ gdb ./ex4
(gdb) break 11
(gdb) run
(gdb) print somme
$1 = 27
(gdb) print somme / 2
$2 = 13
(gdb) next
(gdb) print moyenne
$3 = 13
```

`somme` vaut bien 27, mais `somme / notes.size()` est une **division entière** (un `int` divisé par un entier non signé) : le résultat est 13, et

c'est seulement ensuite qu'il est converti en `double`. La correction consiste à convertir avant la division :

```
double moyenne = static_cast<double>(somme) / static_cast<double>(notes.size());
```

Avec `-Wconversion`, le compilateur aurait d'ailleurs signalé cette ligne.

- # Exercice 5 — Interpréter un rapport

Interprétez ce rapport et proposez une cause probable :

```
ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010
READ of size 4 at 0x502000000010 thread T0
  #0 in afficher(int*) gestion.cpp:8
  #1 in main gestion.cpp:15
freed by thread T0 here:
  #0 in operator delete(void*)
  #1 in main gestion.cpp:14
```

Corrigé

Le programme lit 4 octets (un `int`) dans une zone mémoire **déjà libérée**. La mémoire a été libérée par un `delete` à la ligne 14 de `main`, puis, ligne 15, `main` appelle `afficher` qui lit cette zone à la ligne 8. Cause probable :

```
int* p = new int{42};
delete p;              // ligne 14
afficher(p);           // ligne 15 : p est un pointeur pendant
```

C'est un comportement indéfini. Les pointeurs intelligents (chapitre 14) rendent ce type d'erreur beaucoup plus difficile à commettre.

- # Exercice 6 — Git

Écrivez la suite de commandes pour : créer un dépôt, faire un premier commit de tous les fichiers, créer une branche `calcul-ponderé` pour ajouter une fonctionnalité, y faire un commit, puis fusionner cette branche dans `main`.

Corrigé

```
git init
git add .
git commit -m "Créer la structure initiale du projet"

git switch -c calcul-pondere
# ... modifications ...
git add src/moyenne.cpp include/bulletin/moyenne.hpp
git commit -m "Ajouter le calcul de la moyenne pondérée"

git switch main
git merge calcul-pondere
```

> Remarque : on évite les accents et les espaces dans les noms de branches, d'où `calcul-pondere`.

- # 2.19 Travaux pratiques — TP 2 : mettre en place l'environnement

Objectif : disposer d'un environnement complet et fonctionnel, qui servira pendant tout le cours.

- # Partie A — Installation et vérification

i. Installez l'environnement selon votre système (section 2.3).
ii. Relevez les versions de `g++`, `clang++`, `cmake`, `ninja`, `gdb` et `git`.
iii. Compilez et exécutez le programme de test de la section 2.3.4 avec GCC **et** Clang.

- ## Partie B — Configuration de l'éditeur

  i. Installez VS Code et les extensions de la section 2.5.2 (ou configurez CLion).
  ii. Vérifiez que clangd fonctionne : une erreur volontaire doit être soulignée pendant la frappe.

- ## Partie C — Débogage et sanitizers

  i. Reproduisez la session GDB de la section 2.8.3 sur `bug.cpp`.
  ii. Compilez `bug.cpp` avec ASan et UBSan, et identifiez la ligne fautive dans le rapport.
  iii. Corrigez le bug et vérifiez que le rapport disparaît.
  iv. Analysez `fuite.cpp` (section 2.9.3) avec LeakSanitizer, puis avec Valgrind (sur un binaire compilé **sans** ASan). Comparez les deux rapports.

- ## Partie D — Squelette UIN-Campus

  Réalisez l'étape du fil rouge décrite à la section 2.20, puis :

  i. configurez et construisez le projet avec le preset `debug` ;
  ii. exécutez-le ;
  iii. lancez clang-format et clang-tidy sur tout le projet ;
  iv. créez le dépôt Git, faites le premier commit, et publiez-le sur GitHub.

- ## Livrable

  Le lien vers le dépôt GitHub, dont le `README.md` indique les versions des outils relevées en partie A et les commandes de construction.

- # 2.20 Fil rouge — Squelette du projet UIN-Campus

- ## 2.20.1 Arborescence

```
uin-campus/
├── CMakeLists.txt
├── CMakePresets.json
├── .clang-format
├── .clang-tidy
├── .gitignore
├── README.md
├── include/
│   └── uincampus/
│       └── version.hpp
├── src/
│   └── version.cpp
├── app/
│   └── main.cpp
└── tests/              (vide pour l'instant, chapitre 23)
```

- ## 2.20.2 Le code

`include/uincampus/version.hpp`

```hpp
#pragma once

#include <string_view>

namespace uin {

std::string_view version();

} // namespace uin
```

src/version.cpp

```cpp
#include "uincampus/version.hpp"

namespace uin {

std::string_view version()
{
  return UIN_VERSION;   // macro fournie par CMake
}

} // namespace uin
```

app/main.cpp

```cpp
#include "uincampus/version.hpp"

#include <print>

int main()
{
  std::println("UIN-Campus — Système de gestion académique");
  std::println("Version {}", uin::version());
}
```

La macro `UIN_VERSION` n'est écrite nulle part dans le code : CMake la définit à partir de la version déclarée dans `project(...)`. La version n'existe ainsi qu'à **un seul endroit**.

- ## 2.20.3 `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.25)

project(UINCampus
  VERSION 0.1.0
  DESCRIPTION "Système de gestion académique de l'UIN"
  LANGUAGES CXX)

set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

option(UIN_ACTIVER_SANITIZERS "Activer AddressSanitizer et UBSan" OFF)

# ---------------------------------------------------------------
# Options communes à toutes les cibles
# ---------------------------------------------------------------
add_library(uincampus_options INTERFACE)
target_compile_features(uincampus_options INTERFACE cxx_std_23)

if(MSVC)
  target_compile_options(uincampus_options INTERFACE /W4 /permissive-)
else()
  target_compile_options(uincampus_options INTERFACE
      -Wall -Wextra -Wpedantic -Wshadow -Wconversion)
endif()

if(UIN_ACTIVER_SANITIZERS AND NOT MSVC)
  target_compile_options(uincampus_options INTERFACE
      -fsanitize=address,undefined -fno-omit-frame-pointer)
  target_link_options(uincampus_options INTERFACE
      -fsanitize=address,undefined)
endif()
```

```
# ---------------------------------------------------------------
# Bibliothèque métier
# ---------------------------------------------------------------
add_library(uincampus_core
  src/version.cpp)

target_include_directories(uincampus_core PUBLIC ${PROJECT_SOURCE_DIR}/include)
target_compile_definitions(uincampus_core PRIVATE UIN_VERSION="${PROJECT_VERSION}")
target_link_libraries(uincampus_core PRIVATE uincampus_options)

# ---------------------------------------------------------------
# Application
# ---------------------------------------------------------------
add_executable(uin-campus app/main.cpp)
target_link_libraries(uin-campus PRIVATE uincampus_core uincampus_options)
```

Choix de conception :

- la **bibliothèque** `uincampus_core` contiendra toutes les classes métier (`Etudiant`, `UE`, `Note`…) ; l'**application** ne fera que les utiliser. Cette séparation permettra au chapitre 23 de tester la bibliothèque sans passer par `main` ;
- les options communes sont regroupées dans `uincampus_options` et ne sont écrites qu'une fois ;
- les sanitizers sont une **option**, activée par le preset `debug`.

2.20.4 `CMakePresets.json`

```
{
  "version": 6,
  "configurePresets": [
      {
          "name": "debug",
          "displayName": "Debug avec sanitizers",
          "generator": "Ninja",
          "binaryDir": "${sourceDir}/build/debug",
          "cacheVariables": {
              "CMAKE_BUILD_TYPE": "Debug",
              "UIN_ACTIVER_SANITIZERS": "ON"
          }
      },
      {
          "name": "release",
          "displayName": "Release",
          "generator": "Ninja",
          "binaryDir": "${sourceDir}/build/release",
          "cacheVariables": {
              "CMAKE_BUILD_TYPE": "Release"
          }
      }
  ],
  "buildPresets": [
      { "name": "debug",   "configurePreset": "debug" },
      { "name": "release", "configurePreset": "release" }
  ]
}
```

- ## 2.20.5 Construire et exécuter

```
cd uin-campus
cmake --preset debug
cmake --build --preset debug
./build/debug/uin-campus
```

Résultat attendu :

```
UIN-Campus — Système de gestion académique
Version 0.1.0
```

Puis, pour clangd :

```
ln -s build/debug/compile_commands.json .
```

- ## 2.20.6 Qualité et versionnement

Ajouter à la racine les fichiers `.clang-format` (section 2.10.1), `.clang-tidy` (section 2.10.2) et `.gitignore` (section 2.12.5), puis :

```
clang-format -i include/uincampus/*.hpp src/*.cpp app/*.cpp
clang-tidy -p build/debug src/*.cpp app/*.cpp

git init
git add .
git commit -m "Créer le squelette du projet UIN-Campus"
git remote add origin https://github.com/<utilisateur>/uin-campus.git
git push -u origin main
```

Le projet est prêt. Au chapitre 7, nous y ajouterons les premières classes métier : `Etudiant`, `UE` et `Note`.

2.21 Pour aller plus loin

- Documentation officielle de CMake : tutoriel *CMake Tutorial* sur cmake.org.
- Craig Scott, *Professional CMake: A Practical Guide*.
- Documentation de clang-tidy : liste complète des vérifications sur clang.llvm.org.
- Documentation de GDB : *Debugging with GDB* sur sourceware.org.
- *Pro Git*, de Scott Chacon et Ben Straub, disponible gratuitement en français sur git-scm.com.

Chapitre suivant : ***Chapitre 3 — Les bases du langage en style moderne***